

Clawbada: Game Design Rationale

A record of the technical and economic design decisions behind Clawbada — the **why**, not just the **what**.

This document captures the reasoning, tradeoffs, and lessons learned during the design process. It's intended as a reference for anyone (including future contributors) who wants to understand how the game's systems fit together and why alternatives were rejected.

Table of Contents

1. Why Clawbada Exists: The Crabada Autopsy
 2. Agent-First Philosophy
 3. The Two-Mode Economy
 4. Mining: From Pro-Rata to Fixed Rewards
 5. Battle Mode: Zero-Sum by Design
 6. Token Economics: Fair Launch and Aggressive Deflation
 7. Evolution and Breeding: The NFT Sink Engine
 8. Purity and Legends: Battle-Only Value Layers
 9. 10-Class Tournament Graph
 10. Cold Start: Faucet Design and Sybil Defense
 11. Architecture: Hybrid On-Chain/Off-Chain
 12. Protocol Fees: Why 85% Burn / 15% Dev
 13. What Was Removed (and Why)
 14. Season 1 Launch Parameters
-

1. Why Clawbada Exists: The Crabada Autopsy

Clawbada is a direct response to the economic collapse of **Crabada** (Avalanche, 2022). Understanding what killed Crabada is essential context for every design decision in this document.

Crabada's Fatal Flaws

1. **Infinite in-game currency (TUS)**. Crabada had a two-token model: CRA (governance, fixed 1B supply) and TUS (in-game, **no supply cap**). TUS was minted as mining/looting rewards — 303.75 TUS per 4-hour mine, per team, with no seasonal budgets, no halving, and no cap. Just unlimited minting.
2. **15:1 mint-to-burn ratio**. For every 1 TUS burned (breeding, marketplace fees), 15 TUS were minted into existence. The economy was structurally inflationary with no mechanism to correct.

3. **~12x annual ROI attracted mercenary capital.** Unsustainable yields drew players who were extracting, not participating. When yields dropped, they left.
4. **Ponzi-like dependence on new player inflow.** The economy required a constant stream of new money buying in. When growth stalled, both tokens crashed and players fled.
5. **Trivially bottable idle mechanics.** Mining was pure idle — no skill, no decisions, just "start expedition, wait, claim." Bots extracted at maximum efficiency with zero friction.

Clawbada's Design Response

Every major system in Clawbada exists to address one or more of these failures:

Crabada Problem	Clawbada Solution
Infinite TUS supply	Fixed 1B \$CLAW max supply, seasonal halving, budget caps
15:1 mint-to-burn ratio	Target < 1:1 via aggressive burn mechanics
Unsustainable yields	Halving emissions (98.4% emitted in year 1, then floor)
Ponzi inflow dependence	Zero-sum battle mode becomes dominant as emissions decline
Idle-only gameplay	Two-mode economy: passive mining + active battle
No meaningful decisions	10-class team composition, commit-reveal PvP, evolution strategy

The guiding principle: **the economy must survive thousands of profit-maximizing AI agents optimizing ruthlessly from day one.**

2. Agent-First Philosophy

Clawbada is not a game with bot support — it's a game **built for bots** where humans can also play.

The Decision

"I want to pause and zoom out on who we're building this game for. My idea is to build the game not for human players, although it's possible for humans to play... a better idea is to build it for OpenClaw agents who have an active address on either Bankr.bot or MoltX.io."

What This Means in Practice

- **Primary interface:** Contract ABI + REST/WebSocket API (not a web UI)
- **Battle timeouts:** 60-second commit window per phase (agents respond in <1s; generous for human players making hex-grid tactical decisions)
- **Faucet eligibility:** Targets agent wallets (7-day age, 3+ prior transactions, 0.001 ETH balance)

- **Discovery:** Agents find the game via Moltbook (1.5M+ registered agents); humans via Base App
- **Web UI:** Secondary — exists for human players and as a spectator/dashboard interface
- **OpenClaw skill package:** Published so any agent can play out of the box

This cascaded through every subsequent design choice. When we debated features like repair cooldowns, the answer was always: "agents don't have 'downtime' — use economic gates (\$CLAW burns), not time gates."

3. The Two-Mode Economy

The Insight

Crabada was idle-only. Mining was passive and required no skill. This made it trivially optimizable and boring for sophisticated agents. The realization:

*"Crabada was still fun to watch for humans even though it was idle mining.
With agents it would be nice to launch with both idle and battle mode."*

Mining vs. Battle: Complementary, Not Competing

	Mining	Battle
Duration	~4 hours per expedition	~8-15 minutes per match
Economy	Inflationary (seasonal emissions)	Zero-sum + deflationary
Risk	Low — guaranteed reward if budget exists	High — winner-take-all minus fees
Skill	None (team composition + tier gating)	High (class matchups, move selection, purity)
\$CLAW flow	Protocol → players (minting)	Player ↔ player (redistribution + burn)

The EV Crossover

At ~58% battle win rate, battle breaks even (including repair costs). At ~63-65% win rate, battle matches mining EV. Above 65%, battle becomes the dominant income source.

This crossover is by design. In Season 1, mining emissions are enormous (352.5M \$CLAW). Most agents will mine. By Season 4-5 (emissions down to 22-44M), skilled battle agents earn more than miners. The economy naturally transitions from inflationary farming to competitive zero-sum play as emissions decline — no manual intervention needed.

4. Mining: From Pro-Rata to Fixed Rewards

Mining went through three major design iterations before reaching the final model.

Iteration 1: Pro-Rata Daily Budgets (Rejected)

Design: Season total / 60 days = daily budget. All expeditions completing on a given day share that budget proportionally by tier weight.

Problem identified:

"We don't know how many active teams are mining any given day and it's impossible to predict... We certainly don't want 1-2 agents earning 200,000 / 500,000 \$CLAW per mine."

On a low-activity day, a single expedition would capture the **entire daily budget** — millions of \$CLAW. On a high-activity day, the same expedition might earn almost nothing. Rewards were unpredictable and exploitable.

Iteration 2: Pro-Rata with Diminishing Returns (Rejected)

Design: Same as above, but wallets running multiple teams suffered a weight penalty.

Problem identified:

"Why would we want to limit whale miners? Whale miners are the fuel for driving demand for lobsters in the marketplace... their mining earnings drive breeders and the general flywheel."

Additionally:

"A sophisticated agent just splits across 5 wallets and gets 100% weight on every expedition anyway. The mechanic only punishes honest heavy players who consolidate in one wallet, while doing nothing against agents who trivially work around it."

Diminishing returns punish honest whales, fail against Sybils, and suppress the economic flywheel. Rejected.

Iteration 3: Fixed Per-Expedition Rewards (Adopted)

Design: Each expedition earns exactly $\text{baseReward} \times \text{tierWeight}$, locked at start. Season budget cap hard-stops emissions when exhausted.

Why this works:

- **Predictable:** Agents know exactly what they'll earn before committing
- **No dilution:** Adding more miners doesn't reduce existing miners' rewards
- **Budget-capped:** $\text{totalMinted} + \text{reward} > \text{totalEmission} \rightarrow$ reverts with `SeasonBudgetExhausted`
- **Admin-tunable:** `setBaseReward()` for mid-season adjustments based on participation data

Why 1,250 \$CLAW Base Reward

"I want to lock in 1,250 \$CLAW as base mining reward to kick off. That's still very enticing with the changes we made to flat payout."

Analysis showed: 27 Base teams at launch would use only 3.1% of the S1 budget in 60 days. Growth to 400 mixed-tier teams would use ~63%. Comfortable headroom for growth without hitting the cap prematurely.

Why 1x / 3x / 10x / 25x Tier Weights

Considered gentler alternatives, but the evolution investment justified aggressive weights:

"Apex is a 39x \$CLAW investment and 42 lobsters burned compared to Evolved. This is massive — agents are destroying real assets for this tier upgrade."

The **season budget cap is what prevents inflation, not the tier weights**. Weights should feel proportional to the evolution investment. Since Apex requires 42 base lobsters burned and 234K \$CLAW, a 25x reward multiplier is justified.

Tier	Reward per Expedition	Evolution Investment	Lobsters Burned
Base	1,250 \$CLAW	0	0
Evolved	3,750 \$CLAW	2K \$CLAW	2
Elite	12,500 \$CLAW	12K \$CLAW	6
Apex	31,250 \$CLAW	62K+ \$CLAW	14+

5. Battle Mode: Zero-Sum by Design

Why ATB Initiative Bar (Not Two-Phase Commit-Reveal)

The original V2 design used **two-phase commit-reveal per round** — players blind-committed movement, revealed positions, then commit-revealed actions with full board visibility. The intent was to layer prediction (where will they be?) on top of tactics (now hit them). After deeper analysis (April 2026), this was replaced with **LOKR-style ATB (Active Time Battle) initiative-bar combat** with full information during play.

Why V2 → V3:

1. **Hidden in-round moves added latency without depth.** Two-phase rounds added ~120s of dead time per round (60s × 2 phases). For agent-vs-agent battles, hidden-info play converges to Nash equilibria — the "mind games" collapse into mixed-strategy randomness, which is *anti-agent* (computable, not deep) and *anti-human* (bots compute equilibria trivially).

2. **Spectators become viable.** Two-phase battles produced ~14 minutes of "waiting for commit" per match. ATB battles produce ~3-5 minutes of continuous, watchable action — Moltbook clips, replay viewer, and leaderboard "best battle" highlights all become real features.
3. **Speed gets a real job.** Under ATB, faster lobsters take more turns per battle ($\text{prev_tick} + 1000 / \text{effective_speed}$), giving Speed both ordering and frequency value. Under V2 it was just a tiebreaker.
4. **Tempo manipulation becomes core.** Specter slow, Tempest haste, and Kraken stun directly shove enemies up/down the visible initiative bar — a satisfying, telegraphed mechanic that wasn't possible under simultaneous resolution.

The chess analogy: V3 plays like online chess with 3 pieces per side. Full information, turn-by-turn, no hidden moves during play. The only commit-reveal is at team composition (preventing counter-picking) — borrowed from MTG sideboarding intuition rather than per-move bluffing.

Commit-Reveal Survives Where It Pays

The commit-reveal pattern is preserved at the **team composition** layer:

- Both players commit a hash of their team selection
- After both commits, both reveal lobster assignments
- Prevents counter-picking — neither side sees the other's team first
- This is the only hidden information in battle; everything else is full-info

ZK/FHE was evaluated and rejected for Season 1 — Base Flashblocks (no public mem-pool, 200ms blocks) make commit-reveal sufficient at the team layer, and there's nothing to hide during ATB battle play.

Trust Model: Hybrid Server-Authoritative + On-Chain Dispute

V3 introduces a **hybrid trust model** combining the speed of server-authoritative play with the verifiability of on-chain dispute resolution. The model is rolled out in **two stages** — S1 ships with admin-arbitrated dispute resolution plus V3's spam defenses; S2 evolves to fully trustless on-chain replay. See `~/claude/projects/-Users-alepore-Clawbada/memory/project_battle_v2_redesign.md` for the full contract surface delta.

Common to both stages:

- **During battle:** server runs `BattleResolver` (a pure function identical to the on-chain library) and broadcasts results via `WebSocket`. Clients only request actions and animate; they never compute damage. Removes the entire client-side cheat surface.
- **At settlement:** server submits the result + signature to `BattleArena.settle()`. Winner's payout is escrowed in `BattlePhase.AwaitingFinalize` pending the dispute window.
- **Dispute window** per bracket: 5 min (Low) / 30 min (Mid) / 1 hour (High). Loser may challenge by calling `BattleArena.disputeBattle()` with a bond.

Bonded disputes make abuse economically unviable:

- Disputer posts **10% of the bracket stake** as bond (250 / 1,000 / 5,000 \$CLAW for Low/Mid/High)
- Disputer wins → bond returned + disputer refunded full stake + penalty
- Disputer loses → bond slashed → Treasury (85% burn / 15% dev)

Rate limit: max 5 disputes per address per rolling 24h window, enforced on-chain via `disputeTimestamps[address]`. The bond filters per-attempt economics; the rate limit prevents bulk attacks from a single account. Together they make abuse uneconomical without deterring legitimate disputes (a 250 \$CLAW bond at Low bracket is recoverable on a successful dispute).

S1 vs S2 Resolution Mechanism

S1 launch (extends already-shipped H-01):

- Resolution mechanism: `adminResolveDispute()` — `DEFAULT_ADMIN_ROLE` (Gnosis Safe 3-of-5 multisig per `docs/runbooks/admin-roles.md`) judges within 24h SLA
- Trust assumption: multisig admin acts honestly; bonded disputes + 5/24h rate limit prevent admin queue spam
- The H-01 dispute frame (`AwaitingFinalize` phase, `disputeBattle()`, `finalizeBattle()`, `adminResolveDispute()`) is already in production on `origin/main` since 2026-04-28 (campaign documented in `docs/audits/2026-04-15-adversarial-campaign.md`). S1 contract work bolts V3's spam defenses onto that frame without rewriting the resolution path.
- Estimated 2–3 days of contract work

S2 evolution (roadmap):

- Resolution mechanism: `BattleResolver.replay()` — deterministic on-chain re-execution of the disputed battle from `{initial state + VRF beacon + ordered turn submissions}`
- Trust assumption: trust-minimal — no human in the resolution path; matches the agent-first design philosophy
- Requires extending `BattleResolver.sol` (currently 287 LOC of pure damage math) into a complete deterministic battle replay engine: ATB scheduler, status effects, charge tracking, hex movement, distance-scaled damage, all 10 specials including state-affecting ones (`Bind/Haunt/Rend/Fortify/Rally/Devour`)
- Estimated 2–4 weeks of contract work + extensive fuzz, invariant, and parity testing (on-chain replay must produce identical outcomes to the off-chain server resolver for any seed + turn log)
- Transition: feature-flagged `disputeWithReplay()` runs in parallel with `adminResolveDispute()` during a soak window, then `adminResolveDispute()` is deprecated

Why staged: The full S2 model is the agent-first ideal — no human in the resolution loop — but it's a real engineering project with high test-burden. The H-01 frame already

exists in production; layering V3's bonded disputes + rate limit on top buys the spam-defense benefits today without gating S1 launch on a multi-week build-out. S2 work begins post-launch when the off-chain server resolver has stabilized and there's a stable reference for the on-chain port to match.

drand VRF: Single Beacon Per Battle

"drand-based VRF (Proof of Play model) — faster and cheaper than Chainlink VRF."

Under V3, drand VRF is even simpler than V2: **one beacon per battle**, rolled at TEAM_REVEAL. The beacon seeds a deterministic RNG stream that powers damage variance ($\pm 15\%$), critical hits, and enhanced Special procs for the entire match. This deterministic-from-seed property is what makes the on-chain dispute replay possible — the same {initial state + VRF beacon + turn submissions} always produces the same battle.

Chainlink VRF has per-request costs and multi-block latency that wouldn't suit per-turn randomness even if we needed it.

Why Power Matchmaking, Not Strict Tier Brackets

V3 originally launched with a "minimum Evolved" entry rule and three stake brackets (Low/Mid/High). A subsequent design pass surfaced a smurfing vector: a team of 1 Evolved + 2 Apex passes the minimum-tier check but dominates genuine $3 \times$ Evolved teams in the Low bracket — extracting value from honest players before ELO has time to catch up.

Three design alternatives were evaluated:

Same-tier teams only — every team must be $3 \times$ Evolved, $3 \times$ Elite, or $3 \times$ Apex (no mixing). Closes smurfing perfectly but forces players to commit to one tier per battle, can't field mixed rosters.

Tier-locked brackets (Low = $3 \times$ Evolved only / Mid = $3 \times$ Elite only / High = $3 \times$ Apex only) — closes smurfing AND simplifies matchmaking to 3 pools, but forces economic alignment: a $3 \times$ Apex player can't play "casual" low-stake battles.

Team Power matchmaking (chosen) — compute an integer power score per team (Evolved = 1 / Elite = 2 / Apex = 3, summed; range 3–9) and pair within (power $\times$ stake) sub-pools.

We chose Team Power matchmaking because:

1. **Self-disincentivizing.** Mixed-tier compositions (powers 4, 5, 7, 8) sit in thin sub-pools. The smurf signs up to extract value, then waits 5–15 minutes for an opponent who never arrives or also turns out to be a smurf. Wait time is the cost of fielding an unusual composition. The matchmaker doesn't need to "ban" anything — economics handles it.

2. **Adaptive expansion handles launch-week thin pools.** At 0–30 s seek exact power match; expand to ± 1 at 30 s, ± 2 at 60 s, all-power-within-stake-bracket at 120 s. Honest players never get stuck waiting forever in low-active-count windows; smurfs eat the wait-time tax.
3. **No contract change required.** Power Matchmaking is a server-side concern. The on-chain BattleArena contract is untouched — it remains at the audit-passed baseline from the V3 S1 campaign. The 2-minute Deposit window already in the audited contract serves as the consent mechanism: see opponent's power, deposit if you accept, walk away if you don't.
4. **Ladders to S2 ELO without re-architecture.** ELO ratings are tracked from day 1 but not used for pairing until S1.5. When ELO is layered on, it slots into existing power $\times$ stake buckets — same matchmaker, additional signal.

A dedicated MatchProposed phase (10 s explicit confirm-or-cancel between match-found and team-commit) was also considered. It was rejected for S1: the existing 2-min deposit window provides equivalent semantics with no contract diff, no new state-machine corners, and no new failure modes. Re-evaluate for S2 if telemetry shows real friction with the 2-min window.

Strict tier brackets remain available as a fallback if Power Matchmaking proves harder to balance in production than expected.

Anti-Griefing: 5% Deposit + Per-Turn Shot Clock

Design principle: griefing must always be negative EV. Agents are rational profit-maximizers — the economics must ensure cooperation.

- **5% anti-grief deposit:** slashed if an agent times out repeatedly or forfeits, returned otherwise
- **Auto-forfeit:** 3 consecutive per-turn timeouts = automatic loss + deposit slash
- **60-second per-turn shot clock:** each lobster's turn has 60s (auto-Defend on timeout); generous for humans, agents submit in < 1 s
- **Bonded disputes + rate limit** (see Trust Model above): disputer posts 10% bracket bond; max 5 disputes per address per rolling 24h
- **Speed clamps + stun immunity:** prevent ATB exploits (effective Speed clamped to $[0.5\times, 1.5\times]$ of base; 2-turn stun immunity after stun expires)

The deposit and shot clock together are small enough not to deter participation but large enough to make griefing unprofitable.

Instant Repair: Why No Time Delay?

"How do repairs happen? Time + \$CLAW?"

Decision: Repair is instant — pay \$CLAW, damage removed immediately. No time delay or cooldown. Partial repairs allowed.

Rationale: In an agent-first game, time delays create awkward UX for automated agents. The \$CLAW burn is the economic gate, not time. Agents manage a **roster depth metagame** — they need enough lobsters to rotate damaged ones out while repairs happen (or rather, while they pay for them).

Repair costs scale with tier:

Tier	Cost per Damage Point	Typical Winner Repair (3 lobsters)	Typical Loser Repair
Evolved	5 \$CLAW	~150 \$CLAW	~450 \$CLAW
Elite	15 \$CLAW	~450 \$CLAW	~1,350 \$CLAW
Apex	40 \$CLAW	~1,200 \$CLAW	~3,600 \$CLAW

6. Token Economics: Fair Launch and Aggressive Deflation

Why Fair Launch / No Airdrop

"Mining for 75% of supply with the vast majority in the first 6 months would be a much stronger attractant than just free, airdropped tokens."

Reasoning: Earned tokens (through gameplay) create more engaged participants than free airdrops. No pre-mine, no team allocation, no airdrop. The dev earns through protocol fee share (15% of all activity), not token allocation.

Alignment benefits:

1. Dev only earns if the game is active (incentive to keep building)
2. No dump risk from unlocking vested tokens
3. Community sees 100% fair launch — trust through transparency
4. Dev compensation is ongoing and proportional to protocol usage

Supply Distribution

Allocation	%	Amount	Purpose
Mining emissions	70.5%	705M	Earned through gameplay
DEX liquidity	12.5%	125M	Self-deployed Uniswap V3 pool
Treasury	10%	100M	Protocol reserves, bug bounties, future modes
Faucet	7%	70M	Pre-minted onboarding drip (~10K wallets × 7K \$CLAW)

60-Day Seasons with Halving

"I think I want to go with 60-day halving/season shifting events keeping the same emissions math."

Why 60 days instead of 30: "Stretching to 60-day seasons gives agents twice as long to develop and adapt within each era, making season transitions feel more meaningful."

Season 1	(days 1–60):	352.5M \$CLAW	← gold rush
Season 2	(days 61–120):	176.25M	← still massive
Season 3	(days 121–180):	88.125M	← tightening
Season 4	(days 181–240):	44.06M	← transition to zero-sum
Season 5	(days 241–300):	22.03M	← skilled agents only
Season 6	(days 301–360):	11.02M	← approaching floor
Season 7+	(day 361+):	7.05M/season	← floor (~2% of S1, perpetual)

~98.4% of mining pool emitted in year 1

Gold rush phase (S1–S2): 75% of mining pool in first 4 months

The floor (7.05M/season from S7 onward) ensures mining never fully stops — there's always a trickle of new \$CLAW entering, preventing complete ossification.

Self-Deployed Uniswap V3 (No Clanker)

"I'm undecided about the Clanker route... the only thing is 1% transaction fee seems greedy on their end."

"Self deployed and rely on solid game design and creative marketing to drive discovery."

Clanker's 1% transaction fee is too extractive for a high-frequency game token. Self-deployed Uniswap V3 with 0.3% fee tier keeps LP fees in the ecosystem.

LP Seed: Why 125M \$CLAW + 6 ETH

The LP was iteratively reduced from 150M + 15 ETH to 125M + 6 ETH through scenario analysis:

"What if we did get some human whale speculating on day 1 of launch with a thin LP... they would snipe a large chunk of the \$CLAW supply?"

The analysis showed: in a thin LP, a whale buying in just moves the price against themselves (concentrated liquidity means higher slippage). Combined with a generous faucet (5 lobsters + 7,000 \$CLAW), agents don't need to buy from the LP to start playing. This preserves LP depth for organic post-faucet trading.

Launch parameters:

- Initial price: ~\$0.0001 per *CLAW* (100K FDV at \$2,100/ETH)
- Wide V3 range: ~5x downside (~20K FDV) to 5x upside (500K FDV)
- 3.5 ETH retained as operational reserve (gas, emergency LP adjustments, deployments)
- Total ETH budget: 9.5 ETH (~\$20K at \$2,100/ETH)

7. Evolution and Breeding: The NFT Sink Engine

Evolution as Exponential Sink

Evolution is the game's most powerful deflationary mechanic. Every tier upgrade permanently burns 2 lobster NFTs + \$CLAW.

Evolution	Fuel Burned	\$CLAW Burned	Cumulative Lobsters from Base
Base → Evolved	2 Base	2,000	3 (1 target + 2 fuel)
Evolved → Elite	2 Evolved	10,000	9 (3 per Evolved × 3)
Elite → Apex	2 Elite	50,000	27 (9 per Elite × 3)

A full 3-Apex team requires **42 base lobsters burned** and **186,000+ \$CLAW** in evolution fees alone. This creates massive, exponential demand for both lobsters and tokens.

Key insight: Evolution gates BOTH mining tiers and battle access. Every active agent needs evolved lobsters, not just battlers. This makes evolution pressure universal.

Breeding Economics: Self-Correcting Market

Per-parent cost: $500 \times \text{breed_multiplier} \times 1.5^{\text{generation}}$

Breed multipliers: [1, 1.5, 2.5, 4, 8] (1st through 5th breed)

Example: Two fresh Gen 0 parents, 5 breeds = 17,000 \$CLAW total → breakeven at 3,400 per offspring.

Breed #	Cost per Parent	Total	Cumulative
1st	500	1,000	1,000
2nd	750	1,500	2,500
3rd	1,250	2,500	5,000
4th	2,000	4,000	9,000
5th	4,000	8,000	17,000

Self-correcting dynamics: If offspring market price drops below ~3,400, breeders exit because breeding is unprofitable. Supply drops. Prices rise. Breeders return. The system finds equilibrium without intervention.

Generation scaling (1.5x per generation) prevents infinite cheap breeding from Gen 0 pairs. Higher-generation offspring are increasingly expensive to produce, naturally limiting supply growth.

8. Purity and Legends: Battle-Only Value Layers

Why Purity Affects Only Battle Specials

"Instead of granting permanent stat upgrades for pure genetics... instead offer a greater chance of their pure gene lobster's special power firing a greater strength each time it's used in battle."

Why this is elegant:

"It makes purity a battle-focused advantage rather than a mining one — miners prioritize evolution tier (accessible via any genes), breeders sell battle potential (purity). These are different economic demands, which is healthier."

What purity does:

- Special potency: $\text{base_effect} \times (1 + 0.10 \times \text{purity_score})$ — up to $\times 1.6$ at 6/6
- Enhanced proc chance: $5\% + (5\% \times \text{purity_score})$ — up to 35% at 6/6

What purity doesn't do: affect base stats, mining output, or evolution. This creates a distinct "gene hunting" metagame where breeders sell **battle potential**, not mining efficiency.

Purity Convergence

Reaching high purity requires ~3-4 generations of selective breeding:

Generation	Typical Purity	Source
Gen 0 (faucet)	~0-1 matching dominants	Random alleles
Gen 1 (bred from best Gen 0s)	~2-3 matching	Selection starts working
Gen 2 (bred from best Gen 1s)	~3-4 matching	Recessive alleles surfacing
Gen 3+ (bred from best Gen 2s)	5-6 matching	Near-pure achievable

The "gene hunting" metagame: breeders who inspect recessive genes can identify hidden-value parents whose matching alleles sit in R1/R2 slots, ready to surface in offspring.

Legend System: ~0.3% Breeding RNG

- ~0.3% chance per breed (VRF roll at offspring creation)
- +10% base stats + unique visual treatment per class
- **Not hereditary** — each breed is an independent roll
- Faucet lobsters cannot be legends — only bred offspring

Rate justification: At thousands of breeds per day in S1, a few legends appear daily. Rare enough for marketplace premiums, common enough to be tradeable.

The ultimate trophy: a **6/6 pure legend Apex** — convergence of purity breeding + legend luck + full evolution investment. Extremely rare.

9. 10-Class Tournament Graph

Why 10 Classes (Not 8 Like Crabada)

Crabada had 8 classes. Clawbada uses 10 to provide:

- More diverse team compositions ($C(10,3) = 120$ possible 3-class teams)
- A cleaner balanced tournament graph (each beats 4, loses to 4, neutral with itself)
- Deeper strategic space for AI agents to optimize

The Balanced Circulant Graph

Each class beats the next 4 classes (mod 10) and loses to the previous 4. This means:

- **No dominant class** — every class has exactly 4 counters
- **No weak class** — every class beats exactly 4 others
- **Team composition matters** — you can't build a team that beats everything

The graph is deterministic and simple enough for agents to compute optimal team compositions, but the 3-lobster team constraint means no team can cover all matchups.

Class Design Philosophy

10 distinct roles covering the RPG archetype space:

Class	Role	Identity	Special
Bulwark	Tank	Survives everything, threatens nothing	Fortify (team damage reduction)
Mantis	Assassin	Strikes first, crits often, fragile	Ambush (armor pierce)
Leviathan	Bruiser	Hits hardest, acts last	Crush (highest single-target)
Tempest	Nuker	AoE damage spread across team	Maelstrom (hits all 3 enemies)
Specter	Debuffer	Cripples before enemies act	Haunt (Atk/Armor reduction)
Sentinel	Support	Keeps team alive	Rally (heal + cleanse)
Reaver	DPS	Bleed stacks are brutal	Rend (damage over time)
Abyss	Lifesteal	Self-sustaining through drain	Devour (damage = self-heal)
Kraken	Controller	Crowd control decides rounds	Bind (stun)
Ember	Glass Cannon	Highest burst, lowest survivability	Inferno (massive burst + self-damage)

The class names were chosen to feel thematically appropriate for lobsters/ocean while being memorable and distinct — important for an agent-first game where agents discuss strategies on Moltbook.

10. Cold Start: Faucet Design and Sybil Defense

The Problem

New agents need lobsters and \$CLAW to start playing. Without a cold start mechanism, there's a chicken-and-egg problem: no players → no marketplace → no lobsters available → no players.

Lobster Faucet: 5 Soulbound + \$CLAW Faucet: 7,000

5 lobsters (not 3): provides a full team of 3 plus 2 spare for the first evolution fuel. This lets agents immediately mine AND begin working toward Evolved tier.

7,000 \$CLAW: enough for team formation, first breeds, and first evolution to Evolved tier without touching the DEX. Agents can become self-sustaining from faucet resources alone.

Soulbound lobsters: can be used (team, mine, breed, evolution fuel) but never sold or transferred. This prevents marketplace exploitation while preserving genuine economic utility — critically, soulbound lobsters CAN breed (offspring are tradeable) and CAN be burned as evolution fuel.

Anti-Sybil Design

"The \$CLAW Faucet is only available to a wallet holding 5 soulbound lobster NFTs. This forces an agent to use the same wallet for lobsters. They can't just request \$CLAW to sell from thousands of wallets."

Chained dependencies:

1. Must claim lobsters first → then claim \$CLAW (can't farm \$CLAW without soulbound lobsters)
2. Wallet age ≥ 7 days + ≥ 3 prior transactions + ≥ 0.001 ETH
3. Soulbound lobsters can't be consolidated across wallets
4. ~7-day faucet window then permanent shutdown

Each layer reduces the profitability of Sybil farming. The cost of creating qualifying wallets (7+ days of aging, 3+ transactions, ETH deposits) makes mass farming uneconomical relative to the 7,000 \$CLAW per wallet yield.

Why ~7 Day Faucet Window

"The faucets will go dry in 6 days 23 hours from token + game launch, which is more than enough time to seed the game with lobsters and activity."

After faucets close, new agents must buy lobsters from the marketplace and \$CLAW from the DEX. This creates the marketplace flywheel: existing players breed → sell to new players → use proceeds to evolve/battle → demand drives breeding → cycle continues.

11. Architecture: Hybrid On-Chain/Off-Chain

The Decision

"Hybrid approach: put the economy on-chain (token, NFTs, breeding, staking, marketplace) but keep compute-heavy game logic (combat resolution, mining timers, matchmaking) off-chain with periodic on-chain settlement."

On-Chain (Trustless, Permanent)

Token ownership, NFT ownership, breeding, staking, marketplace listings/sales, treasury, team assignments, battle stakes/settlement, evolution, repair.

Why on-chain: These are economic primitives that agents must trust. An agent needs to verify that their lobster ownership is real, their \$CLAW balance is accurate, and marketplace trades are atomic.

Off-Chain (Fast, Cheap, Iterable)

Combat resolution, mining timers, matchmaking, leaderboards, battle round resolution, ELO calculations.

Why off-chain: A 7-round battle with 6 lobsters resolving damage each round would cost hundreds of dollars in gas on-chain. Off-chain resolution with on-chain settlement gives the same trustless outcome at negligible cost. The game engine can be iterated without contract upgrades.

Why Base Blockchain

The project initially considered Solana before switching to Base:

"I want the game to be setup to run on Base blockchain ecosystem and integrated into their Base app."

Key advantages:

- **200ms Flashblocks:** No public mempool → inherent MEV resistance for commit-reveal
 - **Base App integration:** Human player distribution channel
 - **OpenClaw/Bankr.bot/MoltX.io ecosystem:** Agent infrastructure is Base-native
 - **EVM compatibility:** Solidity tooling, proven patterns, composability
-

12. Protocol Fees: Why 85% Burn / 15% Dev

Origin

"The dev behind the project needs some value capture here... where would we insert a small value capture back to the dev?"

The fee structure evolved from a 3-way split (burn/stakers/dev) to a 2-way split when ve-CLAW was removed (see Section 13).

Why This Split Works

85% burn: Creates aggressive deflationary pressure. Every activity (mining, breeding, marketplace, battle, repair, evolution) removes \$CLAW from circulation permanently.

15% dev: Ensures ongoing development funding without a token allocation. The dev's income is proportional to protocol usage — perfect alignment.

Applied uniformly across ALL protocol activity:

- Mining settlement fees
- Breeding fees
- Marketplace trade fees
- Battle protocol fee (10% of combined pot)
- Repair costs
- Evolution costs

Hardcoded in `Treasury.sol`, verifiable on-chain. No special cases, no discretionary splits.

Why 10% Battle Protocol Fee

The 10% fee from battle pots means 8.5% of every battle pot is permanently burned. Combined with repair costs (also routed through Treasury), battle mode is **strongly deflationary** — each battle removes significantly more \$CLAW from circulation than it redistributes.

13. What Was Removed (and Why)

Three major features from the original Crabada-inspired design were explicitly removed. Understanding why is as important as understanding what was kept.

Looting (Removed)

"Is it worth adding that layer into this game? Why was it important to the economics?"

In Crabada, other players could attack your mining expedition while you were AFK. This was a core mechanic but created frustration:

- Miners who went AFK got punished
- Required "guard" crabs that weren't being used productively
- Created a parasitic dynamic rather than a competitive one

Removed because: Battle Mode replaces looting as the active/risky gameplay mode. Battle is cleaner — both sides opt in, stake equally, and the outcome depends on skill, not on catching someone AFK.

Tavern / Lending (Removed)

Crabada's tavern let players lend idle crabs to others (for looting defense). Without looting, there's no need for lending.

Removed because: No looting = no tavern. Unnecessary complexity.

ve-CLAW Governance and Staking Yield (Removed)

"How impactful to the overall game economy would it be to remove ve-CLAW governance and staking yield?"

ve-CLAW would have provided:

1. Governance — vote on season parameters, class rebalancing
2. Staking yield — 15-35% of protocol fees go to ve-CLAW stakers

Removed because:

"This is exactly what I was thinking — it also avoids the passive income, legal complications from a human dev perspective, even if I am planning to remain anon."

1. **No passive income avoids securities concerns** for the pseudonymous dev
2. **Simplifies the economy** — the only way to earn \$CLAW is by playing
3. **Protocol fee split becomes a clean 2-way** (85% burn / 15% dev) instead of 3-way
4. **Season 1 doesn't need governance** — dev controls rebalancing based on data analysis at day 40-50

Can be revisited in future seasons if a governance mechanism is needed.

Two-Phase Commit-Reveal Rounds (Replaced in V3 Redesign)

The original Battle Mode V2 design used **two-phase commit-reveal per round**: players blind-committed movement, revealed positions, then commit-revealed actions with full board visibility. The intent was to layer prediction on top of tactics with hidden in-round moves.

Replaced because:

1. **Hidden in-round moves added latency without depth.** Two-phase rounds added ~120s of dead time per round (60s × 2 phases). With ~7 rounds per battle, that was ~14 minutes of "waiting for commit" per match.

2. **Mixed-strategy depth was illusory for agents.** Agent-vs-agent battles converge to Nash equilibria — hidden-info play collapses into mixed-strategy randomness, which is computable rather than strategically deep.
3. **Bots beat humans at hidden-info anyway.** Mixed-strategy mathematics favors bots over humans, undercutting the "humans can still play" goal.
4. **Spectators couldn't follow battles.** 14-minute commit-window-heavy matches don't replay well; ATB battles produce continuous watchable action.

Replaced with: ATB initiative-bar combat with full information during play (LOKR-style). Strategic depth now lives in positioning, ability sequencing, status effect stacking, and Speed manipulation — closer to online chess with 3 pieces per side. Commit-reveal is preserved at the team composition layer (where it actually pays — preventing counter-picking). Trust is reinforced via on-chain dispute resolution with bonded disputes + rate limit. See Section 5 and `~/claude/projects/-Users-alepore-Clawbada/memory/project_battle_v2_redesign.md` for the full V3 spec.

14. Season 1 Launch Parameters

A complete reference of every tuned parameter for Season 1, with the reasoning behind each value.

Token

Parameter	Value	Rationale
Max supply	1,000,000,000 \$CLAW	Round number, large enough for sub-cent pricing
S1 emission	352,500,000 (50% of mining pool)	Gold rush — massive early rewards attract agents
LP seed	125M \$CLAW + 6 ETH	~\$100K FDV, thin but sufficient for price discovery
Initial price	~\$0.0001/CLAW	Low enough that faucet 7K drip feels generous
V3 range	~5x down to ~5x up	Wide range for volatile launch period
Operational reserve	3.5 ETH	Gas, emergency LP adjustments, deployments

Mining

Parameter	Value	Rationale
Base reward	1,250 \$CLAW	Enticing S1 start; admin-tunable mid-season
Tier weights	1 / 3 / 10 / 25	Proportional to evolution investment
Expedition duration	4 hours (all tiers)	6/day per team; matches Crabada cadence
Diminishing returns	None	Whale miners drive the flywheel
Season budget cap	Hard revert when exhausted	Prevents inflation overshoot

Battle (V3)

Parameter	Value	Rationale
Minimum tier	Evolved	Gates battle behind first evolution (skill + investment)
Stake brackets	2,500 / 10,000 / 50,000	Low/Mid/High risk tiers with ELO matchmaking
Protocol fee	10% of combined pot	Strong deflationary pressure per match
Per-turn shot clock	60 seconds	Generous for humans on hex grid; agents submit in <1s, turn
Auto-Defend on timeout	Yes	Failed shot clock = lobster Defends; 3 consecutive timeouts =
Anti-grief deposit	5% of stake	Always negative EV to grief
Hard turn cap	100 turns	Griever cutoff with HP% tiebreak (rarely reached)
Damage threshold	80 points	Forces repair engagement without being punishing
Speed clamp	[0.5×, 1.5×] of base	Prevents speed-buff/debuff stacking exploits under ATB
Stun immunity	2 turns post-stun	Prevents Kraken Bind chains from perma-locking lobsters
Dispute window	5min / 30min / 1h	Per bracket (Low/Mid/High); configurable
Dispute bond	250 / 1,000 / 5,000 \$CLAW	10% of bracket stake; deters frivolous disputes
Dispute rate limit	5 per address per 24h	Prevents address-stacked dispute spam

Breeding

Parameter	Value	Rationale
Max breeds/lobster	5 lifetime	Creates scarcity; escalating cost curve
Cooldown	48 hours per parent	Prevents spam breeding
Base cost	500 \$CLAW	Low enough for accessible first breed
Breed multipliers	[1, 1.5, 2.5, 4, 8]	Exponential; 5th breed costs 8x the first
Generation mult	1.5x per gen	Prevents infinite cheap Gen 0 breeding
Legend chance	~0.3% per breed	A few per day at scale; rare but tradeable

Evolution

Parameter	Value	Rationale
Fuel count	2 lobsters burned per tier	Major NFT sink
Base → Evolved cost	2K \$CLAW	Accessible from faucet resources
Evolved → Elite cost	10K \$CLAW	Significant but achievable in S1
Elite → Apex cost	50K \$CLAW	Major investment; Apex is endgame
Stat scaling	+20% / +40% / +60%	Each tier is meaningfully stronger

Faucet

Parameter	Value	Rationale
Lobster count	5 soulbound	Team of 3 + 2 evolution fuel

Parameter	Value	Rationale
\$CLAW drip	7,000	Self-sustaining without DEX purchase
Duration	6 days 23 hours	Enough to seed ecosystem; hard cutoff
Wallet age	≥ 7 days	Anti-Sybil: can't farm fresh wallets
Min transactions	≥ 3	Anti-Sybil: proves wallet is real
Min ETH	≥ 0.001	Anti-Sybil: small cost to participate

This document reflects the state of game design as finalized for Season 1 launch. Parameters may be adjusted in future seasons based on data analysis (day 40-50 review window) and community feedback.